Pyrmit – technical report

Data, Swedish legal context, pipeline and evaluation methodology. Companion to the benchmark report. Generated on 2026-10-24; figures are read from the data and result files at generation time.

1. Purpose of the system
2. The Swedish planning system and where Pyrmit fits
3. Data sources: what, why, how
4. Keeping the data current
5. Pipeline, with code references
6. Retrieval evaluation
7. Evaluation: how the benchmarks are built and scored
8. Results summary
9. Limitations and roadmap

Appendix: commands and files

1. Purpose of the system

Pyrmit answers, in English, whether a building or land project is feasible in a given Swedish municipality and how to get it approved: which rules decide the case (national law, the municipality's plans and local guidance), what permits are needed, and what remains uncertain. Every rule, number or deadline in an answer must cite a passage the system actually read; citations open the original document at that passage. It is a **pre-application screening** tool: it does not replace the municipality's decision, and the answers say what must be checked.

2. The Swedish planning system and where Pyrmit fits

2.1 Who decides what

- **Municipal planning monopoly.** Planning the use of land and water is a municipal matter (PBL 1 kap. 2 §). The municipality's building committee (byggnadsnämnd) decides on permits; its planning bodies adopt plans.
- **Comprehensive plan (översiktsplan, PBL 3 kap.)**: covers the whole municipality, states intentions (where to build, what to protect). Not binding, but it guides permit and planning decisions.
- **Detailed plan (detaljplan, PBL 4 kap.)**: binding for an area: land use (residential, roads, nature…) and property provisions (building area, ridge height, storeys, minimum plot size, land that may not be built on, design, implementation period). It consists of a *plankarta* (map with codes and provisions) and a *planbeskrivning* (explanatory description). Plans go through consultation (samråd), review (granskning), adoption (antagande) and gain legal force (laga kraft). Area regulations (områdesbestämmelser) are a lighter binding instrument.
- **Permits (PBL 9 kap.)**: building permit (bygglov), demolition permit (rivningslov), site permit (marklov); some measures only need a notification (anmälan) or are exempt (e.g. Attefall measures). Inside a detailed plan the project must comply with the plan (small deviations can be accepted); outside, the site is assessed against the general suitability requirements of PBL 2 kap. An advance ruling (förhandsbesked) can settle the location question first. **Note**: PBL chapter 9 was renumbered by SFS 2025:974 (bygglov conditions now 9 kap. 56–57 §, small deviations 60–61 §, advance ruling 74 §); case law and most guidance still use the old numbers.
- **Environmental Code (miljöbalken)** constraints that often decide feasibility: shoreline protection (strandskydd, MB 7 kap.; building needs an exemption with a special reason), productive agricultural land (MB 3 kap. 4 §: only for essential public interests), areas of national interest (MB 3–4 kap.).
- **Water and sewage**: municipal water and sewage areas (LAV); a private sewage system needs a permit or notification to the municipal environmental committee (förordningen om miljöfarlig verksamhet och hälsoskydd, 13–14 §§).
- **Property formation** (fastighetsbildning) is decided by Lantmäteriet under the Real Property Formation Act; detailed plans can set minimum plot sizes.
- **Appeals**: building committee decisions are appealed to the county administrative board (länsstyrelsen), then to a Land and Environment Court (mark- och miljödomstol), then to the Land and Environment Court of

Appeal (Mark- och miljööverdomstolen, MÖD), which needs leave to appeal and sets precedents. Decisions to adopt a detailed plan are appealed directly to the Land and Environment Court.

2.2 Where Pyrmit fits

Question in a project	Legal basis	What Pyrmit reads
Is my site inside a detailed plan, and what does it allow?	PBL 4 kap., plan provisions	Plan registry, plan maps and descriptions crawled from the municipality (tools `plans`, `plan_rules`, `read`)
Do I need a permit, a notification, or nothing?	PBL 9 kap., PBF	Statute text (`law_section`), municipal permit pages
Can I build outside a plan / get an advance ruling?	PBL 2 kap., 9 kap.	Statute text, comprehensive plan, municipal guidance
Shoreline, farmland, national interests?	MB 3, 4, 7 kap.	Statute text, comprehensive plan, municipal pages on strandskydd
Water and sewage, fees, timelines?	LAV, FMH, municipal fee schedules (taxa)	Municipal pages and PDFs (sewage guidance, taxa)

The system mirrors how a planning officer screens a case: identify the municipality and the site, check the plan situation, find the national rules that apply, then the local documents. It stops before what needs official data it does not have yet (which plan covers a given property, maps of protected areas) and says so in the answer.

3. Data sources: what, why, how

Source	What we take	Why this source	How (code)	Terms
Riksdagen open data (data.riksdagen.se)	15 consolidated statutes (PBL, PBF, MB, JB, FBL, LAV, FMH…)	Official consolidated text with amendment markers; free API	`corpus/laws.py` (`LAWS`, `main`)	Open data, cite Riksdagen
Wikidata (SPARQL)	290 municipalities: official code, county, website; ~10k localities	Stable identifiers and official websites; localities for place → municipality	`corpus/municipalities.py` (`sparql`, `pick_site`)	CC0
Municipal websites	30,953 pages, 70,926 PDFs (182.5 GB): permits, sewage, fees, plans, comprehensive plans	Local rules and detailed plans are published by each municipality; no national full-text source	`corpus/crawl.py` (`content_relevance`, `crawl`)	Public web pages; robots.txt honoured, 1 request/s per host
Local OCR (PP-OCRv6, GPU)	Text and word positions of the scanned PDFs (mostly old plan maps): 305 of 6 read so far	Old plans have no text layer	`corpus/ocr.py`	Local processing
Plan registry (derived)	31,777 plans, 7,782 with 75,238 extracted provisions	Agent needs plans as entities with status and rules	`corpus/plans.py` (`plan_key`, `provisions_of`, `status_of`)	Derived
Case law: MÖD via lagen.nu	746 rulings since 2011, 390 on our topics	Real decisions with facts, outcome and reasons: ground truth for the benchmark	`corpus/caselaw.py` (`parse_case`, `relevant`)	Court rulings are public documents without copyright protection; lagen.nu allows the ruling pages in robots.txt

3.1 Statute versions in the corpus

Statute	Consolidated up to	Downloaded
Anläggningslag (1973:1149)	SFS 2022:362	2026-10-06
Bostadsrättslag (1991:614)	SFS 2026:847	2026-10-06
Expropriationslag (1972:719)	SFS 2019:850	2026-10-06
Fastighetsbildningslag (1970:988)	SFS 2020:921	2026-10-06
Förordning (1998:899) om miljöfarlig verksamhet och hälsoskydd	SFS 2026:1515	2026-10-06
Jordabalk (1970:994)	SFS 2026:1498	2026-10-06
Kulturmiljölag (1988:950)	SFS 2024:813	2026-10-06
Lag (2006:412) om allmänna vattentjänster	SFS 2022:1249	2026-10-06
Lag (1973:1150) om förvaltning av samfälligheter	SFS 2026:846	2026-10-06
Ledningsrättslag (1973:1144)	SFS 2021:742	2026-10-06
Miljöbalk (1998:808)	SFS 2026:1442	2026-10-06
Miljöbedömningsförordning (2017:966)	SFS 2026:706	2026-10-06
Miljöprövningsförordning (2013:251)	SFS 2026:943	2026-10-06
Plan- och byggförordning (2011:338)	SFS 2026:1722	2026-10-06
Plan- och bygglag (2010:900)	SFS 2026:1583	2026-10-06

3.2 Crawl

290 municipalities crawled on 2026-10-23 (163,690 requests). The crawler starts from each official website, follows links scored by topic keywords (building permits, plans, sewage, fees; negative keywords for unrelated sections), keeps pages with a relevance score ≥ 4 (`KEEP_SCORE`), PDFs linked from relevant pages, and caps each municipality (1000 pages, 3000 PDFs, 40 MB per PDF, 15000 requests, depth 4, per-section quotas). Text is extracted with PyMuPDF block by block, repeated headers and footers removed (`corpus/extract.py` `page_blocks`, `strip_boilerplate`), each document classified (`doc_type`: detaljplan 37,414, other 15,069, bygglov 9,288, avlopp 9,161, oversiktsplan 8,439, taxa 3,042, strandskydd 604).

Corpus served by the app: 1,880,149 chunks in 83,032 documents, 288 municipalities, 3,651 statute sections. Plan statuses in the registry: in_force 15,874, unknown 9,577, in_progress 3,571, consultation 1,254, review 967, adopted 534.

3.3 Sources not used yet (and why they matter)

- **Lantmäteriet**: the property register (property designations, boundaries) and national access to digital detailed plans. Needed to answer "which plan covers my property" reliably. Access terms and APIs to be confirmed (some are paid).
- **Svensk författningssamling** (svenskforfattningssamling.se): the official publication of new acts; useful to detect amendments before the consolidated text is updated.
- **County boards and national agencies**: GIS layers for shoreline protection, national interests, protected nature; Boverket's guidance (PBL kunskapsbanken). Would turn spatial questions ("is my plot within 100 m of the shore?") into data lookups.
- **Municipal plan services** (WMS / plan registers): many municipalities publish plan boundaries as map services; combined with an address or property, they give the applicable plan.

4. Keeping the data current

4.1 Current state (honest picture)

- Statutes: the consolidated version is visible in the raw text ("t.o.m. SFS …", table 3.1) but not stored as metadata; texts with two versions of a section (in force now / from a future date) are both indexed.
- Municipal documents: **no fetch date or HTTP validators are stored per page or PDF** (the crawl manifest keeps id, URL, title, score); the only date is the crawl itself (2026-10-23–2026-10-23). Fee schedules change yearly; plan pages change when plans move through the process.
- Plans: status is inferred from page URLs and document text (laga kraft date, "antagandehandling"…); superseded or repealed plans are not tracked.
- Rebuilds are manual: extract → chunk → embed (incremental) → store → plan registry. Chunk ids include a text hash, so unchanged chunks keep their vectors.

4.2 How to guarantee freshness

1. **Provenance on every document**: fetched_at, HTTP ETag / Last-Modified, content hash, source URL, and for statutes the "t.o.m. SFS" version and the date of entry into force of each section version. Shown in the source panel and in the answer ("municipal page fetched on …").
2. **Scheduled refresh by volatility**: statutes weekly (Riksdagen API, compare the version marker; check svenskforfattningssamling.se for new acts); municipal plan and permit pages weekly, the rest of the crawl monthly, with conditional requests (If-None-Match / If-Modified-Since) and sitemaps so unchanged pages cost one cheap request; case law monthly.
3. **Incremental pipeline**: re-extract and re-chunk only changed documents, embed only new chunk ids, update the store in place (insert / delete rows and vectors) instead of rebuilding; rebuild the plan registry for touched municipalities.
4. **Legal validity**: index each statute section with its validity interval (the consolidated text marks future versions "Träder i kraft I:…" and expiring ones "Upphör att gälla U:…"), answer with the version in force on the question date; track plan status changes (adopted → legal force → amended / repealed).
5. **Prefer authoritative sources** where they exist (Lantmäteriet's digital plans and property data) over crawled copies, and keep the crawl for what has no national source (local guidance, fees).
6. **Monitoring and regression**: after each refresh, a report of changed / failed sources per municipality, and a benchmark run (MÖD sample) to catch quality regressions; alert when a municipality's site structure changes (crawl yields drop).

5. Pipeline, with code references

Stage	What it does	Code
Municipalities	Wikidata SPARQL → codes, websites, localities	`corpus/municipalities.py`
Statutes	Riksdagen API → raw text per SFS	`corpus/laws.py`
Crawl	Async, robots.txt, 1 req/s/host, topic scoring, caps, resumable manifests	`corpus/crawl.py`
OCR	PP-OCRv6 medium (RapidOCR, DirectML), tiled pages, word boxes, plans first	`corpus/ocr.py`
Extraction	PyMuPDF blocks, boilerplate removal, doc types, OCR text when needed	`corpus/extract.py`
Chunking	One chunk per statute section (chapters only on their own line, lettered §, transitional provisions dropped); ~1,500-char document chunks; corpus-wide dedup; ids = hash(doc, index, text)	`corpus/chunk.py` (`law_chunks`, `document_chunks`)
Embeddings	Snowflake arctic-embed-l-v2 (1024 dim) on the GPU, incremental	`localrag/index.py`, `embeddings.py`
Disk store	SQLite (chunks, FTS5 BM25 on Swedish stems with pool / municipality filter tokens) + float16 vectors (memmap → GPU)	`localrag/store.py` (`build`, `DiskStore`, `DiskRetriever`)
Plan registry	Plans, status, provisions tied to chunks	`corpus/plans.py`
Place → municipality	Wikidata names and localities ∪ corpus place names	`localrag/gazetteer.py`
Agent tools	find_kommun, search (hybrid RRF), grep, list_documents, read, law_section, plans, plan_rules, view_map_area (also on OCRed scans), view_pdf_page; search / grep / list_documents filter on document types and title, grep can list the matching documents; every output tagged with [c:chunk_id]	`agentic/tools.py` (`CorpusTools`, `TOOL_SCHEMAS`)
Agent loop	Responses API tool loop: exploration turns at `explore_effort`, parallel tool calls, `answer_ready`, streamed final answer at `effort`, step budget	`agentic/agent.py` (`run_agent`, `SYSTEM_PROMPT`)
Chat integration	Background thread, SSE events, live citation numbering, saved trace	`agentic/service.py` (`stream_agent`, `CitationNumberer`), `routers/chat.py`
Source viewer	Cited chunk → PDF page with highlight, or URL text fragment	`routers/docs.py`, `frontend/components/SourceViewer.tsx`

Grounding by construction. Tool outputs carry chunk ids; the answer format requires a [c:id] for every rule, number, fee or deadline, and forbids stating what was not read in a tool result. The chat replaces ids by numbers while streaming and attaches, for each number, the cited chunks (pipeline event `citations`).

5.1 Technical choices that shape the answers

Each choice below limits what the agent can find or say. They are listed with their current setting (read from the code and the crawl manifests when this report is generated), the reason, and the visible effect, so that a wrong or missing answer can be traced to one of them.

Stage	Choice	Setting (code)	Why	Effect on the answers
Crawl	**Page selection**	Keyword score on URL + title ≥ 4 (`KEEP_SCORE`, `_KEYWORDS`); news, schools, care, culture, minutes and e-services skipped (`_NEGATIVE`); links followed on the same score, but a negative word in a long link text does not hide an on-topic section (`link_relevance`); the home page's site index (Webbkarta, A–Ö) is followed	Municipal websites are mostly off-topic; a fixed budget per municipality. Menu cards mix topics ("Bygglov, … detaljplaner, … livsmedel") and some menus are built in JavaScript	Rules published on a page whose URL and title carry no planning word are missing; council minutes and local decisions are not in the corpus, so the agent has no local precedents
Crawl	**Budget per municipality**	1000 pages, 3000 PDFs, 40 MB per PDF, 15000 requests, depth 4 (pages at the last depth still give their PDF links); one site section ≤ 1/5 of the pages	1 request/s per site: the budget sets the crawl time (one night for the country). Large municipalities publish 1,000–3,000 PDFs; the section quota keeps a plan archive from using the whole budget	2 municipalities at the PDF cap, 0 at the request cap: their remaining plans are missing and the agent reports "no plan found" for places that have one
Crawl	**Which PDFs**	Linked from a relevant page, with a topic word or a plan hint in the link; only obvious noise is skipped (`off_topic_pdf`: annual reports, alcohol licences, estate notices…, unless a plan word), and each skipped link is recorded; one copy per URL (http/https, www) and per content hash	A word list cannot judge relevance: meeting notices carry planning decisions, waste rooms are a building-permit topic, template titles lie ("Kallelse" on a planuppdrag decision). The real decision is the relevance filter below, with a second opinion	Skipped by this crawl: duplicate 11660, off topic 427, too big 628
Filter	**PDF relevance**	Rules on title, file name and link text (plan words win), then a logistic regression on arctic vectors of those fields + the first 700 characters; every proposed drop goes to MiMo Flash ("keep when in doubt") or a manual review (`corpus/pdf_filter.py`, `pdf_review.jsonl`)	The large crawl brings municipal administration (HR policies, fees under the Environmental Code, flag rules…). Losing a useful document costs more than keeping noise: the LLM kept ~1/3 of the drops proposed by the rules or the model	7,920 PDFs dropped (7.1% of the PDF text), each confirmed by the LLM or a review; borderline documents (energy plans, committee packs) stay
Crawl	**Politeness and failures**	robots.txt (with Google's * and $), 1 request/s per host, a user agent naming the project; a municipality that returns nothing keeps its previous crawl	Public sites, no load on them; a site down on the crawl day must not empty a municipality	Municipalities without data: Gällivare, Karlskrona (site blocking this crawler, down or unreachable)
Crawl	**What a crawler cannot reach**	HTML links only: no JavaScript, no map or search application	A browser-based crawl is slower and heavier; map services need one integration per vendor	Sites whose navigation is built in JavaScript (Jönköping, Lidingö's plan documents) and municipalities that publish their plans only in a map or search application (11 of 29 sampled for the coverage benchmark, benchmark report 2.11) are covered only through the pages that link documents directly. Next: the national plan register or the municipal map services (section 3.3)

Extraction	**PDF text**	PyMuPDF text layer, repeated headers / footers removed; < 150 characters per page = scanned, OCR by PP-OCRv6 medium (recognition cut to the Latin characters, detection input 1280 px; text pages at 300 dpi and maps at 200, word boxes kept), by tier: 1 plan maps and provisions (40 pages); 2 plan texts (20 pages); other planning types, studies attached to plans, other on request; no topic filter here (the relevance filter decides)	Free on the local GPU; best of the engines measured on old plan scans (OCR benchmark: 87 % of the words and 8 of 13 property labels read on map crops, 69 % and 0 for EasyOCR); word boxes let the agent find a property label on a scanned map	10,531 scanned PDFs without OCR have no text: invisible to search and grep. Most are old plans, whose rules are printed on the map itself, and old planbeskrivningar. Tables and map legends lose their layout
Extraction	**Document type**	First matching keyword in URL, link text, title, start of text (`doc_type`)	Cheap, transparent	A `doc_types` filter in search / grep / list_documents can miss a misclassified plan; a scan without OCR has no text to classify on, so many old plan documents are typed "other"
Chunking	**Statutes**	One chunk per § (lettered § separate), a chapter only on its own line; transitional provisions dropped; a § with a future version is indexed twice	A § is the unit lawyers cite; references wrapped after "kap." must not open a chapter	The agent cannot cite transitional rules (which version applies to an application filed before an amendment); it may cite a version not yet in force
Chunking	**Documents**	~1,500 characters on paragraph boundaries, a short previous paragraph repeated, header = municipality + title + section heading; identical chunks kept once in the whole corpus	Fits the embedding model; standard plan wording would otherwise flood the results	A rule split over two chunks needs two reads; standard wording is citable from one plan only (~2 % of plan provisions point to another chunk of the same map)
Retrieval	**Search**	Snowflake arctic-embed-l-v2 on the Swedish query + BM25 on Swedish stems, fused (RRF), k = 8 (max 15), 350-character snippets; `read` returns up to 9,000 characters	Best recall on the retrieval eval (section 6); BM25 brings exact names and numbers	The agent sees snippets first and must read to get the full rule; plan numbers and property labels are found by grep / BM25, not by the embedding
Retrieval	**Place → municipality**	Wikidata names and localities ∪ place names found in the corpus; the municipality named outright first	No property register (Lantmäteriet) yet	A place name shared by two municipalities can send the search to the wrong one ("Kårsta": Vallentuna and Örebro)
Agent	**Model and budget**	GPT-6 Luna, exploration and answer at medium effort, at most 10 steps, the last one forced to `answer_ready`	Chosen on the benchmark (accuracy first, then cost and time)	A question needing more than ~10 steps is answered with what was found; an image (map, page) is seen for one step only, then only the agent's notes remain
Agent	**Answer format**	Mode chosen by the agent (verdict / explain / propose / develop); every rule cited; "complete answer"	Benchmark report, section 2.9	Long answers; ~1.5 unsupported claims per answer remain (Luna judge)
Plans	**Plan registry**	Rule-based: plans grouped by file name, status from URLs and text, provisions from coded lines	No national digital plan register in the corpus	Which map area carries which code is not extracted: per-zone rules need `view_map_area`, which finds the property label in the text layer, or on a scan in the words the OCR located
Serving	**Timeouts**	LLM call 180 s, 1 retry; SSE keep-alive every 10 s; the browser reloads after 45 s of silence	A stalled provider must not hold a worker or freeze the chat	An answer can take a few minutes in the worst case; it is never lost (saved by the worker)

6. Retrieval evaluation

`localrag/eval.py` on `localrag/questions.jsonl`: 45 use-case questions (permits, plans, sewage, fees, heights…), each labelled with the passages that answer it (exact excerpts of municipal documents or statute sections).

Metrics per question: recall@k = share of the expected passages found in the top k; complete@5 = all expected passages in the top 5; MRR = 1 / rank of the first expected passage. The municipality filter is detected from the question. Setups: *dense-en* / *dense-sv* = embedding of the English question / of a Swedish rewrite; *bm25* = BM25 on Swedish tokens; *hybrid* = reciprocal-rank fusion; *+rerank* = cross-encoder bge-reranker-v2-m3.

Model	Setup	R@1	R@5	R@10	Complete@5	MRR
st-arctic-l-v2	dense-sv	0.83	0.94	0.96	0.89	0.87
st-bge-m3	dense-sv	0.78	0.91	0.91	0.84	0.83
or-qwen3-8b	dense-sv	0.71	0.91	0.94	0.84	0.78
st-kblab-swedish	dense-sv	0.51	0.70	0.86	0.60	0.61
st-bge-m3	dense-en	0.32	0.63	0.79	0.53	0.48
or-qwen3-8b	dense-en	0.50	0.78	0.88	0.71	0.62
st-arctic-l-v2	hybrid-sv	0.69	0.87	0.89	0.78	0.75
st-arctic-l-v2	dense-sv +rerank	0.73	0.88	0.93	0.78	0.81
BM25	bm25-sv	0.51	0.76	0.80	0.62	0.62
BM25	bm25-sv-stem	0.46	0.68	0.79	0.56	0.56
BM25	bm25-en	0.00	0.04	0.06	0.04	0.02

Findings: querying in Swedish is decisive (+13 to +28 points of recall@5 over the English query, depending on the model); local open models match API models (arctic-embed-l-v2: 0.92 recall@5); BM25 on English queries is useless on a Swedish corpus; neither BM25 fusion nor the reranker beat dense retrieval on the Swedish query. The agent still uses hybrid search because exact wording (plan names, "20 meter") matters inside its tool calls.

National corpus: the same questions on the corpus of 2026-10-22 and on the national one (arctic-embed-l-v2; *hybrid-sv-stem* = the agent's search, BM25 on Swedish stems). The four municipalities of the questions have three times more passages (18.8k → 59.7k with the statutes); no expected passage left the corpus, but some fall below the top 5 (ranks 7 to 12), outranked by other documents of the same municipality. The statute misses are the same in both corpora: the loss is in the municipal documents.

Setup	Corpus	R@1	R@5	Complete@5	R@5 municipal passages
dense-sv	2026-10-22	0.83	0.94	0.89	1.00
dense-sv	national	0.73	0.88	0.82	0.92
hybrid-sv-stem	2026-10-22	0.61	0.89	0.80	0.97
hybrid-sv-stem	national	0.57	0.83	0.73	0.89
bm25-sv-stem	2026-10-22	0.46	0.68	0.56	0.84
bm25-sv-stem	national	0.41	0.68	0.58	0.76

7. Evaluation: how the benchmarks are built and scored

Four benchmark sets, each built to answer a different question. All cases share one format (one JSON line per case): the `question` as a user would ask it, in English; the `kommun` codes it concerns; the accepted `verdicts` (feasible / conditional / not_feasible / unclear); 3–5 `key_points` an expert answer must make; the `evidence` the system should have read (matchers, section 7.5); and the `source`. They differ in where the scenario comes from and who wrote the reference.

Set	Scenario	Documents it is built from	Reference written by	Size
14 case studies `agentic/cases.jsonl`	Realistic questions (user feedback, typical projects); 3 are real development projects	Detailed plans, municipal guidance pages and statutes of 5 municipalities	Hand, from those documents	14
MÖD rulings `agentic/cases_caselaw.jsonl`	**Real** applications that were refused or granted and appealed to the highest planning court	The rulings of Mark- och miljööverdomstolen (lagen.nu), current statute text for the evidence	LLM, from the ruling (facts, outcome, reasons)	260 generated, 50 sampled
Plankarta `agentic/cases_plans.jsonl`	**Real** plan maps and property numbers; the project is designed	Plan maps (plankartor) in force or adopted, and the plan legend from the plan registry	Vision LLM, from a crop of the map + the legend; only confident labels kept	20
Explain / propose / develop `agentic/cases_tasks.jsonl` `agentic/cases_generated.jsonl`	Questions beyond the verdict: how a procedure works, where a project could go, what an idea is missing (section 7.9)	Municipal guidance pages, comprehensive plans (growth and rural passages), statutes	Hand (10), and MiMo v2.6 Flash with verbatim quotes checked (28)	38

7.1 The 14 case studies (hand-written)

Where they come from. The questions started from the feedback on the first version of Pyrmit (questions on Vallentuna such as “what is the building permit procedure”, “what is the maximum height in Veda”) and were extended to the project types the tool is meant for: building on a detailed plan, Attefall houses, shoreline protection, an eco-village on farmland, private sewage, plot subdivision, rezoning requests, demolition, advance rulings. Five municipalities were chosen because the corpus covers them well: Vallentuna, Uppsala, Norrtälje, Ydre and Gotland.

Real or constructed. Three cases are real development projects on real detailed plans (Kumla-Stensta, Mörby backe and Kristineberg in Vallentuna): the question asks about a project slightly beyond, or within, what the adopted plan allows, and the plan's own provisions decide. The others are realistic scenarios built on the municipality's real pages and the statutes. None is an application with a known decision: the reference is what the documents say, written by hand, and every key point is backed by an **exact excerpt** of the document or a statute section (the expected evidence).

Case	Question	Source documents	Accepted verdicts
`kumla-200m2` ★	I own one of the new plots in Kumla-Stensta, Vallentuna. Can I build a single-storey house with a 200 m2 footprint, plus a 40 m2 garage?	Detaljplan Kumla-Stensta, planbeskrivning (Vallentuna)	not_feasible, unclear
`morby-five-storeys` ★	We are a developer looking at the Mörby backe site (Teknikvägen / Mörbyvägen) in central Vallentuna. Could we build a five-storey apartment block with about 8,000 m2 of gross floor area?	Detaljplan Vallentuna-Mörby (Mörby backe), planbeskrivning	not_feasible, conditional
`kristineberg-terraced` ★	Is it possible to build terraced houses (radhus) in the new Kristineberg district in Vallentuna, and how big can they be?	Detaljplan Kristineberg etapp 1 (Vallentuna)	feasible, conditional
`attefall-vallentuna`	My house is inside a detaljplan area in Vallentuna. Can I build a 25 m2 guest cottage (attefallshus) in the garden without a building permit?	PBL 9 kap. 4 §; Vallentuna page 'Nybyggnad av komplementbyggnad'	feasible, conditional
`ydre-lake-house`	We want to build a holiday house about 40 metres from a lake shore in Ydre, outside any zoning plan. Is that possible?	Miljöbalken 7 kap.; Ydre page Strandskydd	conditional, not_feasible, unclear
`ecovillage-vallentuna`	We plan an eco-village with 20 houses, shared gardens and a shared sewage treatment plant on farmland we would buy outside Vallentuna town. What would it take?	PBL, MB, FMH, anläggningslag; Vallentuna planbesked page	conditional, unclear
`uppsala-septic`	I'm building a house in the countryside in Uppsala municipality with no municipal sewer. What do I need for the sewage system, and what does it cost?	FMH 13 §; Uppsala page 'Enskilt avlopp – söka tillstånd eller anmäla'	feasible, conditional
`vallentuna-subdivide`	I own a large plot in Vallentuna and want to split off part of it to sell as a building plot. How does that work?	Fastighetsbildningslagen; Vallentuna lantmäteri page	conditional, feasible
`norrtalje-rezone`	We own farmland near Norrtälje and want it rezoned for about 30 homes. What is the process and how long might it take?	Norrtälje planbesked pages; PBL 5 kap.	conditional, unclear
`ydre-barn-demolition`	Can I tear down an old barn on my farm in Ydre? The farm is not in a zoning plan area.	PBL 9 kap. 43-44 §§; Ydre rivningslov page	feasible, conditional
`norrtalje-permanent-home`	Can I convert my holiday cottage in a fritidshusområde in Norrtälje into my permanent home?	Norrtälje översiktsplan 2050 pages	conditional, unclear
`vallentuna-tower`	Could we build a 30 m high lookout tower on open land outside the built-up area in Vallentuna?	Vallentuna översiktsplan / FÖP	conditional, unclear
`uppsala-advance-ruling`	I'm thinking of buying a field plot outside a village near Uppsala to build a house. Can I find out beforehand whether I'd be allowed to build?	PBL 9 kap. 41-42 §§; Uppsala förhandsbesked page	conditional, feasible

`gotland-own-va`	I want to build a house on Gotland with my own well and septic system. What does Region Gotland require for water and sewage in the building permit case?	Region Gotland page 'Vatten och avlopp i bygglovsärenden'; FMH 13 §	conditional, feasible

★ real project on a real detailed plan

7.2 The MÖD cases (real rulings)

Why real rulings. A ruling of Mark- och miljööverdomstolen is a real application (a building permit, an advance ruling, a shoreline exemption, a detailed plan…) that the municipality decided, that was appealed, and on which the highest court in planning matters gave the final answer with its reasons. It gives what hand-written cases cannot: a real outcome to check the verdict against, decided on the law, and a variety of situations (119 municipalities).

1. **Collection** (`corpus/caselaw.py`): 746 rulings since 2011 from lagen.nu (robots.txt allows the ruling pages; 1 request/s), with headnote, keywords, statutes cited and full text; 390 are on our topics (building permits, detailed plans, shoreline protection, advance rulings, sewage…).
2. **Transformation** (`agentic/cases_from_caselaw.py`, one LLM call per ruling, prompt below): the facts of the ruling become the question, written as the applicant would ask it *before* the decision, with neither the outcome nor the reasoning; the actual outcome gives the accepted verdicts (a remittal or a fact-dependent decision gives “conditional” / “unclear”); the court's reasons give the key points; the statutes it applied give the expected evidence.
3. **Checks**: purely procedural rulings dropped (standing, sanctions); questions that mention the court dropped (`LEAK` regex); municipality names mapped to codes. Evidence = the longest run of ≥ 5 words of the statute wording quoted in the ruling that is found in the **current** statute text, so it survives the 2025 renumbering of PBL chapter 9 and small amendments; 174 of the 260 cases have such a label, the others (rulings under the old PBL) are scored on verdict and key points only.
4. **Sample**: the benchmark runs on 50 cases drawn with a fixed seed, round-robin over the three outcomes (refused, granted, open), labelled cases first (`sample_cases` in `agentic/bench.py`).

Split of the 260 cases by accepted verdicts: not_feasible: 131, feasible / conditional: 70, conditional / unclear: 38, unclear: 8, feasible: 5. Topics (ruling keywords): bygglov 43, detaljplan 33, strandskydd 19, betydande olägenhet 10, tillbyggnad 10, strandskyddsdispens 9, riksintresse 8, förhandsbesked 7. Case-generation prompt (`agentic/cases_from_caselaw.py` `INSTRUCTIONS`):

```
You build test cases for an assistant that tells people whether a building or land project is
feasible in Sweden. You get a ruling of Mark- och miljööverdomstolen (MÖD). Return JSON only:
{
 "usable": true|false,   // false if the ruling is not about whether a concrete project/measure may go ahead
                         // (pure procedure, fees, sanctions, appeal standing, environmental permits for industry).
                         // A förhandsbesked, bygglov, rivningslov, marklov, strandskyddsdispens, change of use or
plan
                         // IS usable whatever the outcome, also when MÖD remitted it after stating the decisive
legal
                         // test (then verdicts ["unclear"] or ["conditional","unclear"]).
 "usable_reason": "<one sentence>",
 "kommun": "<kommun name, e.g. Varberg>",
 "question": "<English, 2-5 sentences, written by the applicant before applying: the project and the facts that
               decided the case as the applicant would know them (place type, distances, existing buildings, what
the
               detaljplan/översiktsplan says, shoreline, farmland...). Never the outcome, the court's assessment or
the
               case number. Do not name the property designation; you may name the kommun and the area.>",
 "outcome": "<one sentence, what MÖD decided>",
 "verdicts": ["feasible"|"conditional"|"not_feasible"|"unclear", ...],  // acceptable verdicts for an assistant
                         // answering the question: permit granted -> ["feasible","conditional"]; refused ->
                         // ["not_feasible"] (add "unclear" only if the facts in the question leave it open)
 "key_points": ["<3-5 English points an expert answer must make, each naming the rule, e.g. 'Outside a detaljplan
                 a new house needs bygglov and a localisation assessment under PBL 2 kap. (PBL 9 kap. 31 §)'>"],
 "statutes": [{"law": "<one of PBL, PBF, MB, JB, FBL, AL, LAV, FMH, KML>", "section": "<as cited in the ruling,
e.g. 9 kap. 31 b §>",
               "quote": "<5-12 consecutive words of the provision's own wording, copied exactly as the ruling
quotes
                         the statute (not the court's paraphrase), e.g. 'avvikelsen är förenlig med detaljplanens
                         syfte'>"}]
                         // 1-4 provisions decisive for the outcome (PBL 2010:900, MB 1998:808, ...). Leave out
                         // provisions of the old PBL (1987:10, ÄPBL) and ones the ruling does not quote.
}
```

7.3 The plankarta cases (real plan maps, designed projects)

Why. In the two other sets the map is never needed (2 map views in 576 agent answers): their answers are in the text of plans and statutes. But a detailed plan's rules often differ by zone (a height or a building area per block), and only the map shows which zone a property is in. This set asks about a specific property where that matters.

1. **Candidates** (`agentic/cases_from_plans.py` `candidates`, rule-based, free): plans in force or adopted from the plan registry, whose plan map has a text layer and at least two coded height or building-area rules (so the rules differ by zone); on the map, a property number (e.g. “5:125”) printed at most twice, whose surroundings carry one of those codes; scale labels, legend and title-block areas and unconfirmed tract names are filtered; at most two per municipality. The crop around the property is saved. 80 candidates.
2. **Reference** (`label`, one vision call per candidate, high effort): the model gets the crop and the meaning of the codes from the plan legend, decides which zone the property is in and which codes apply, and designs a realistic project (house, extension, garage) **within** the limits for half of the cases and **slightly above** one limit for the other half, so that “not feasible” is not a free win. It writes the question in the owner's words, without plan codes, and the key points with each code and value. Only high-confidence labels are kept: 20 cases in 18 municipalities (“not feasible” accepted for 11, not for the other 9).
3. **Evidence** = the plan map document (`doc_id`): the system must have had it in front of it.

Real or constructed. The plan, the map, the property and its codes are real; the project and the question are constructed, and the reference is a model's reading of the map. The crops are kept in `data/agentic/plan_cases/` for a manual check; a sample should be reviewed by a person.

7.4 Example cases

"I own one of the new plots in Kumla-Stensta, Vallentuna. Can I build a single-storey house with a 200 m2 footprint, plus a 40 m2 garage?"

Source: Detaljplan Kumla-Stensta, planbeskrivning (Vallentuna)

Accepted verdicts: not_feasible, unclear. **Expected evidence**: maximeras till 180; 9 kap. 3 §

Expected key points:

- The Kumla-Stensta detaljplan limits the building area (byggnadsarea) per property to 180 m2 in the northern part and 160 m2 in the southern part, so a 200 m2 footprint exceeds it
- A separate ancillary building (komplementbyggnad) of up to 60 m2 is allowed, so the 40 m2 garage fits
- A new house requires a bygglov (PBL 9 kap. 3 §)
- There is no municipal water and sewage: a shared well / wastewater solution is needed

Agent, exploration low / answer medium: verdict "not_feasible" (correct), key points 0.25, evidence 0.50, 12 tool calls, 43 s. *Judge: The answer correctly identifies that the 200 m^2 house exceeds the plot limits, but misses the stated 60 m^2 allowance for an ancillary garage, the building-permit requirement, and the shared water and wastewater solution.*

Agent, exploration medium / answer medium: verdict "not_feasible" (correct), key points 0.75, evidence 0.50, 15 tool calls, 29 s. *Judge: The answer correctly identifies the house-area limit, garage allowance, and need for a house permit, but omits the shared water and wastewater requirement; its claim that the plankarta is legally binding is not supported by the cited excerpt.*

Agent, exploration medium / answer high: verdict "not_feasible" (correct), key points 0.62, evidence 0.50, 12 tool calls, 45 s. *Judge: The answer correctly identifies the house-area limit and garage allowance, but only implies rather than clearly states that a new house requires a permit and omits the shared water and wastewater requirement.*

Classic RAG (medium answer): verdict "not_feasible" (correct), key points 0.50, evidence 0.50, 0 tool calls, 19 s. *Judge: The answer correctly identifies that the 200 m^2 house exceeds the stated limit and the 40 m^2 garage fits the allowance, but omits the new-house permit requirement and shared water and wastewater solution.*

"We want to build a holiday house about 40 metres from a lake shore in Ydre, outside any zoning plan. Is that possible?"

Source: Miljöbalken 7 kap.; Ydre page Strandskydd

Accepted verdicts: conditional, not_feasible, unclear. **Expected evidence**: 7 kap. 15 §; {"any": [{"law": "Miljöbalk", "section": "7 kap. 18 c §"}, {"kommun_code": "0512", "contains": "ansökan om strandskyddsdispens"}]}

Expected key points:

- Shoreline protection (strandskydd) covers 100 m from the shoreline (MB 7 kap. 14 §)
- New buildings are prohibited in the strandskydd area (MB 7 kap. 15 §)
- It is only possible with a strandskyddsdispens from the kommun, which requires special reasons (särskilda skäl, MB 7 kap. 18 c §)
- A bygglov is also needed (and a förhandsbesked can be requested outside detaljplan)

Agent, exploration low / answer medium: verdict "conditional" (correct), key points 1.00, evidence 1.00, 14 tool calls, 34 s. *Judge: The answer correctly explains that a house 40 metres from the shore would generally require a shoreline-protection exemption based on special reasons, as well as building approval, while adding three details not supported by the supplied excerpts.*

Agent, exploration medium / answer medium: verdict "conditional" (correct), key points 1.00, evidence 1.00, 30 tool calls, 37 s. *Judge: The answer correctly identifies the usual 100-metre protection, the building prohibition, the need for an exemption based on special reasons, and building-permit requirements; a few process or approval details go beyond the cited excerpts.*

Agent, exploration medium / answer high: verdict "conditional" (correct), key points 1.00, evidence 1.00, 33 tool calls, 47 s. *Judge: The answer captures the key conditions, but adds a few specific details not established by the reference or cited excerpts.*

Classic RAG (medium answer): verdict "conditional" (correct), key points 0.88, evidence 0.50, 0 tool calls, 15 s. *Judge: The answer correctly identifies the 100-metre protection, building ban and need for an exemption with special grounds, but it understates that a building permit is required and omits the possibility of a preliminary decision outside a detailed plan.*

"I want to convert two existing farm buildings on a rural agricultural property in the Boarp area of Båstad into homes. The buildings were built for agricultural use but have not been used for about 25 years, and the surrounding farmland is leased and farmed. The ground beneath the buildings is gravel and stone, and I understand it has not been classified as agricultural land in the relevant agricultural register."

Source: MÖD 2026:17 (P 13047-25, 2026-05-27): Förhandsbesked för ändrad användning av två ekonomibyggnader för jordbruk – actual outcome: MÖD set aside the municipality's positive preliminary decision and remitted the matter for the municipality to determine whether the land beneath the buildings is valuable agricultural land and, if so, apply the statutory protection test.

Accepted verdicts: unclear. **Expected evidence**: från allmän synpunkt tillfredsställande sätt genom att annan mark tas i anspråk

Expected key points:

- A positive förhandsbesked requires the proposed land use to be suitable under PBL 2 kap. 2 §.
- Under MB 3 kap. 4 §, valuable agricultural land may be used for development only to meet substantial societal interests where the need cannot be satisfactorily met by using other land.
- Existing agricultural buildings do not, by themselves, mean the land beneath them cannot be valuable agricultural land or that changing their use avoids the protection in MB 3 kap. 4 §.
- The municipality must establish whether the land is valuable agricultural land before deciding whether the statutory protection test applies.

Agent, exploration low / answer medium: verdict "unclear" (correct), key points 0.50, evidence 1.00, 14 tool calls, 30 s. *Judge: The answer recognizes unresolved planning and agricultural-land questions, but it does not clearly state the PBL suitability test or the full MB protection test, and it does not explicitly say the municipality must establish whether the land is valuable agricultural land.*

Agent, exploration medium / answer medium: verdict "conditional" (wrong), key points 0.62, evidence 1.00, 37 tool calls, 56 s. *Judge: The answer correctly explains the agricultural-land test but classifies the project too positively because the municipality has not established whether the land is valuable agricultural land or whether the proposal is suitable under PBL.*

Agent, exploration medium / answer high: verdict "conditional" (wrong), key points 0.50, evidence 1.00, 19 tool calls, 48 s. *Judge: The answer classifies the conversion as conditionally feasible, but the reference calls for an unclear outcome because the municipality must first establish whether the land is valuable agricultural land; the claims about durability and adaptation requirements are unsupported by the cited excerpts.*

Classic RAG (medium answer): verdict "conditional" (wrong), key points 0.50, evidence 0.00, 0 tool calls, 25 s. *Judge: The answer states a conditional-feasibility conclusion, gives the agricultural-land test, but omits the PBL suitability requirement and only indirectly addresses the municipality's duty to determine land status.*

"I own an attefall house and an older seasonal cottage in a coastal small-house area in Österåker; their lawful use has been residential accommodation complementary to a permanent home. I am considering renting them to tourists through online booking services for stays of a few days to a week, with a cleaning company cleaning after each stay; in 2024 they were rented about 34 and 20 times respectively, at a similar frequency for several years. Can I use the buildings this way?"

Source: MÖD 2026:13 (P 11254-25, 2026-05-21): Tillsyn med anledning av åtgärder på fastighet – actual outcome: MÖD dismissed the appeal, finding that the rentals required building permission for a change of

use and that the municipality should continue its supervision rather than close the complaint without action.

Accepted verdicts: conditional, unclear. **Expected evidence**: för ett väsentligen annat ändamål än det som byggnaden

Expected key points:

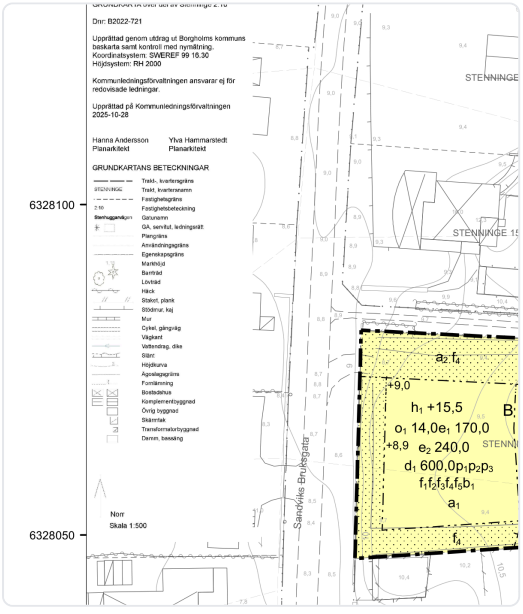
- Under PBL 9 kap. 2 § första stycket 3 a, building permission is required when a building is taken into use for a substantially different purpose from its last lawful use.
- Whether the purpose is substantially different depends especially on the scale of the activity and its effects on the surroundings; use of the whole building may also matter.
- Residential use complementary to permanent living differs from temporary accommodation such as hotel or guesthouse use.
- Repeated paid short-term rentals arranged through booking services, together with cleaning after each stay, can indicate temporary accommodation when assessing the activity's character and scale.
- The ruling establishes that permission is required, not whether an application for permission would be granted.

Agent, exploration low / answer medium: verdict "unclear" (correct), key points 0.60, evidence 1.00, 9 tool calls, 28 s. *Judge: The answer captures the permit test and the relevance of repeated short lets and cleaning, but omits the effects-on-surroundings factor and does not clarify that the ruling establishes a permit requirement, not whether permission would be granted.*

Agent, exploration medium / answer medium: verdict "conditional" (correct), key points 0.80, evidence 1.00, 20 tool calls, 50 s. *Judge: The answer captures the conditional permit risk and the relevance of repeated short stays and cleaning, but omits effects on the surroundings and does not clearly state that the ruling decides only whether permission is required, not whether it would be granted.*

Agent, exploration medium / answer high: verdict "conditional" (correct), key points 0.80, evidence 1.00, 24 tool calls, 48 s. *Judge: The answer correctly identifies a possible change-of-use permit requirement and the relevance of repeated short stays and cleaning, but it does not clearly state that the ruling establishes a permit is required or address effects on the surroundings.*

Classic RAG (medium answer): verdict "unclear" (correct), key points 0.20, evidence 0.00, 0 tool calls, 20 s. *Judge: The answer appropriately flags uncertainty but omits the central permit test and the ruling's indication that repeated short-term rentals with turnover cleaning may constitute temporary accommodation.*



Detaljplan för del av (Borgholm), plankarta, around 15:5

"I own property 15:5 in Borgholm, inside the detaljplan "Detaljplan för del av". I want to build a one-storey house with a 15-degree roof pitch and a 160 m² building footprint. Is that allowed, and what do the plan's rules

allow on my property?"

Source: Detaljplan för del av (Borgholm), plankarta, around 15:5

Accepted verdicts: not_feasible. **Expected evidence**: {"doc_id": "pdf:0885:b8493afd1cf586d5"}

Expected key points:

- o1: the maximum roof pitch is 14.0°, so the proposed 15° pitch exceeds the limit.
- h1: the maximum height is +15.5 m (RH 2000).
- e1: the building-area limit is 170.0 m²; e2 gives 240.0 m² in its applicable scope.
- d1: the minimum property area is 600.0 m².

Agent, exploration medium / answer medium: verdict "conditional" (wrong), key points 0.62, evidence 1.00, 22 tool calls, 92 s. *Judge: The answer correctly notes that 15° exceeds the 14° limit, but classifies the proposal as conditional rather than not feasible and omits the 600 m² minimum property-area rule.*

Agent, exploration medium / answer medium – no view_map_area: verdict "not_feasible" (correct), key points 0.62, evidence 1.00, 28 tool calls, 97 s. *Judge: The answer correctly identifies the roof-pitch violation and area limits, but omits the 600 m² minimum property area and unnecessarily casts doubt on whether 15:5 is within the plan.*

Agent, exploration medium / answer medium – no map reading at all: verdict "not_feasible" (correct), key points 0.62, evidence 1.00, 25 tool calls, 40 s. *Judge: The answer correctly identifies the roof-pitch conflict and area limits, but omits the +15.5 m height cap and 600 m² minimum lot size; it also makes three specific claims not supported by the supplied excerpts.*

Classic RAG (medium answer): verdict "not_feasible" (correct), key points 0.00, evidence 0.00, 0 tool calls, 26 s. *Judge: The answer misses the applicable 14° roof limit and gives unsupported alternative limits and uncertainty instead of the stated rules for the property.*

7.5 Evidence labels and matching

A matcher is a dict checked against every chunk the agent was shown (`localrag/eval.py` `matches`): `law` (substring of the statute title) + `section` or `contains` (verbatim phrase, whitespace- and case-normalised), `kommun_code` + `contains` / `title`, or `any` (alternatives). For MÖD cases, the generator asks for 5–12 words of the statute as quoted in the ruling, then keeps the **longest run of ≥ 5 consecutive words found in the current statute text** (`longest_phrase`): this survives renumbering and small amendments ("tas i anspråk" → "tas i anspråk eller inreds"). Without a match, a section number is used only outside the renumbered PBL 9 kap.; otherwise the case has no evidence label. `check_labels` refuses to run a benchmark whose labels match no chunk.

7.6 Metrics

Metric	Formula	Code
verdict_ok	judge's verdict ∈ accepted verdicts	`judge`, `run_case`
points	Σ judge scores (0/1/2 per key point) / (2 × number of key points)	`run_case`
evidence (seen)	share of matchers satisfied by at least one chunk the system saw (agent: tool outputs; RAG: retrieved chunks), no LLM	`evidence_recall`
evidence cited	share of matchers satisfied by at least one chunk the answer cites ([c:id], truncated ids resolved by unique prefix), no LLM	`agentic/report.py` `reading_metrics`
passages seen	distinct chunk ids in the tool outputs (agent) or retrieved (RAG). A precision "relevant / read" is not computed: the evidence labels list the decisive passages (1–3 per case), not every useful one, so most useful passages would count as irrelevant, and more so for the agent, which reads 3–4 times more	`reading_metrics`
grounded	supported claims / claims, second judge with the full text of the cited chunks	`agentic/grounding.py`
unsupported	claims in neither the reference nor the cited sources (first judge)	`judge`
mode_ok	the agent's `answer_ready(mode)` equals the case's task (explain / propose / develop sets)	`run_case`
anchors	propose: min(places named, `min_anchors`) / `min_anchors`; a place counts when its name or an alias appears as a whole word in the answer, no LLM	`anchor_hits`, `grade_answer`
proposals, proposals_sourced	propose: options the judge finds in the answer, and the share the cited sources (or the reference) support	`grade_answer`
points_obstacle / lever / ask	develop: the key-point score per kind of point	`grade_answer`
seconds, first_token_s, cost	wall time; time to the first streamed answer token; OpenRouter `usage.cost` summed over calls	`agentic/agent.py` `AgentRun`

The judge receives the question, the key points, the accepted verdicts, the answer and up to 15 cited chunks (1,500 characters each). Prompt (`agentic/bench.py` `JUDGE_INSTRUCTIONS`):

```
You grade answers of a Swedish planning assistant against a reference.
Return JSON only: {"verdict": "<feasible|conditional|not_feasible|unclear>", "points": [<0|1|2> per key point, in
order], "unsupported_claims": <int>, "comment": "<one sentence>"}.
- verdict: how the answer classifies the project (Feasible -> feasible, Feasible with conditions -> conditional,
Not feasible as described -> not_feasible, Unclear -> unclear).
- points: 2 = the key point is clearly stated, 1 = partially / vaguely, 0 = missing or contradicted.
- unsupported_claims: number of specific rules/numbers/facts in the answer that are stated neither in the reference
nor
  in the cited sources given below, or that contradict them. A claim the cited sources state is supported even when
  the reference does not mention it (the reference is not exhaustive).
```

Grounding prompt (`agentic/grounding.py` `INSTRUCTIONS`):

```
You check whether an answer is grounded in its cited sources.
List the specific factual claims of the answer (rules, numbers, deadlines, fees, document facts, legal references);
ignore generic advice. For each, decide if the cited sources support it.
Return JSON only: {"claims": <int>, "supported": <int>, "unsupported_examples": ["<short claim>", ...up to 3]}.
```

7.7 Protocol

- Systems: the agent (`run_agent`, efforts set per run) and a classic RAG baseline (`run_rag` in `agentic/bench.py`: the chat's local pipeline, i.e. a rewrite call that also writes a Swedish search query, one dense search on it with

the municipality filter, the top 10 statute sections + top 10 municipal passages, one streamed answer in the same format). Evidence for the RAG = the passages it retrieved.

- Corpus: the full disk store (`--store disk`), so cases from any municipality are answerable.
- Same model for agent and judges (GPT-6 Luna via OpenRouter); judge effort medium. The explain / propose / develop sets are judged by MiMo v2.6 Flash (`--judge-model`), another model family; result rows store `judge_model`.
- **Judge check** (`agentic/rejudge.py`): 60 stored answers graded again by xiaomi/mimo-v2.6-flash: same verdict class in 95 % (kappa 0.89), key points 0.44 vs 0.43 (correlation 0.90, mean gap 0.08); MiMo counts fewer unsupported claims (0.8 vs 2.0). The Luna judge is consistent with another family and, if anything, stricter.
- Repeats (`--repeats`) and merged files give 2–4 passes per configuration; per-pass means measure noise.
- Cases run in parallel (`--workers 3`): latencies include some contention.
- Results are written after every case (`data/agentic/bench-*.json`, one row per answer with the full answer, tool steps and judge comment); `--resume` completes an interrupted run.
- Reports: `agentic/report.py` (summary) and this file, `agentic/report_technical.py`.

7.8 Scope: which questions these benchmarks represent

The three sets ask one kind of question: **a defined project at a known place: is it feasible, under which rules, and how to get it approved?** This is pre-application screening. Within it, the sets are skewed in known ways:

- The place is always given (municipality, often the plan or the property) and the project is specified (use, size, height); the question is single-turn, in English.
- The 14 case studies come from 5 municipalities that the corpus covers well: coverage is better than for an average municipality.
- Appealed rulings are contested cases by selection: more borderline situations and more refusals than ordinary applications, so the MÖD set is harder than typical traffic and refusal-heavy.
- The plankarta cases are designed so that about half the projects slightly exceed a limit.

Not measured, although plausible uses of the tool:

- **Shaping a project from a vague idea** ("I have land near Stockholm and want to build a small eco-village: what could I build, where, and in which order?"): proposing options, the constraints that decide between them, the questions to ask the municipality, over several turns. No set rewards a good design conversation; the verdict / key-point scoring does not fit it.
- Process guidance beyond the key points (documents to submit, sequencing of permits, timelines), and follow-up turns.
- Finding where a project is possible (which municipality, plan or site allows it) and comparing rules across municipalities.
- Questions where the user does not know the property or the plan, and arguments addressed to the municipality or the neighbours.

The scores of these three sets therefore say how well Pyrmit *checks* a defined project, not how well it helps *build* one. Section 7.9 adds sets for explaining, proposing places and developing an idea (single turn); multi-turn guidance, cross-municipality comparison and arguments for the municipality or neighbours are still not measured.

7.9 Beyond the verdict: explain, propose, develop

Why. The product vision (ReGen Villages: help landowners and groups move a project into approval, find where the land allows what, explain the rules) and the user feedback ("what is the building permit process", "how do I change the zoning", "where can I look up zoning") ask more than a verdict on a finished project. A case now has a `task`: **verdict** (the three sets above), **explain** (how a procedure, fee, deadline or map works), **propose** (the place is not fixed: where in the municipality could the project go?) or **develop** (an idea roughly placed: what could block it, how to make it work, what is still unknown).

The agent picks the mode. It classifies the question when it calls `answer_ready(mode)` and answers in that mode's format (a verdict line only for feasibility questions; propose = 2–4 options named in the municipality's plans, each with why it fits, its limits and the first step; develop = obstacles, ways to make it work, suggested path, questions for the user). On the last allowed turn only `answer_ready` is offered, so a run that reaches the step budget still picks its mode. Guide given to the agent (`agentic/agent.py` `MODES_GUIDE`):

```
Kinds of questions (the mode of answer_ready; a question can mix them: pick the main one):
- verdict: a defined project at a known place: is it allowed, under which rules, how to get it approved?
- explain: how a rule, procedure, fee, deadline, map or document works; there is no project to judge.
- propose: the place is NOT fixed: the user asks where in a kommun a project could go (or what could be done with some
  land). This stays propose even when they also ask which approvals or studies it would take. Work fast from the
  översiktsplan: grep the kommun for its growth wording (e.g. 'utvecklingsområde|utbyggnadsområde|nya bostäder|
  prioriterade|serviceort|LIS'), read the 1-3 strongest passages (and its fördjupningar if listed), and note its limits
  (strandskydd, riksintressen, jordbruksmark, VA capacity). Propose places named in those documents, never invented
  ones, with what the documents say about each; do not research every place in depth (4-6 turns).
- develop: the place and the idea are given (roughly) and the user asks what is missing or how to make it work: find
  what could block it and the ways to make it work (another location or scale, phasing, the plan route: planbesked +
  detaljplan vs förhandsbesked + bygglov, liten avvikelse, dispens, shared facilities as a gemensamhetsanläggning),
  and what you need to know from the user.
```

Hand-written seeds (10, `agentic/cases_tasks.jsonl`): Vallentuna (permit process, rezoning, where to look up zoning: the feedback questions; an eco-village near the Roslagsbanan stations; tiny houses near Kårsta), Norrtälje (co-housing: Rimbo, Hallstavik, Älmsta, Bergshamra), Uppsala (rural homes: the prioritised towns; farm housing) and Gotland (plan queue and prioritisation; farm-stay cabins with shared water and sewage). Every key point is backed by an exact excerpt of a municipal document or a statute.

Case	Task	Question	Source
`explain-vallentuna-permit-process`	explain	What is the building permit process in Vallentuna, step by step? When can I actually start building, and how long does the kommun take to decide?	Vallentuna pages 'Följ ditt bygglovsärende', 'Bygglov – ansökan'; PBL 9 kap. 99 §
`explain-vallentuna-rezoning`	explain	I own land in Vallentuna that is not zoned for housing. What is the process to change the zoning so I can build homes there, how long does it take and what does it cost?	Vallentuna page 'Planbesked – ansökan'; PBL 5 kap. 3-5 §§
`explain-vallentuna-zoning-lookup`	explain	Where can I look up the zoning and the rules that apply to a property in Vallentuna, and which map do I need for a building permit application?	Vallentuna page 'Karttjänster'
`explain-gotland-plan-queue`	explain	We would like to get a new detaljplan for about 12 homes on Gotland. How does Region Gotland handle such requests, and how quickly would our plan be worked on?	Region Gotland page 'Ansök om planbesked' (prioriteringsmodell)
`propose-vallentuna-ecovillage`	propose	We are about 15 households who want to build a small eco-village (15-25 homes, shared gardens and workshop) somewhere in Vallentuna kommun, ideally close to public transport. Where in the kommun could this realistically be built, and what would the route to approval be?	Översiktsplan Vallentuna 2040 (bebyggelseutveckling); Vallentuna planbesked page
`propose-norrtalje-cohousing`	propose	Our group (10-15 households) wants to start a co-housing community with a few small workshops in Norrtälje kommun, ideally near a lake. Which parts of the kommun should we look at, and what would we need to get approval?	Norrtälje Översiktsplan 2050 and FÖP Norrtälje stad pages (bostadsförsörjning, strandskydd)
`propose-uppsala-rural-homes`	propose	I'm a developer interested in building a group of about 20 small houses in Uppsala kommun outside the city itself. Where does the kommun want this kind of growth, and what limits apply in the countryside?	Översiktsplan Uppsala 2016, Del A (tätorter, landsbygd)
`develop-karsta-tiny-houses`	develop	We have an idea for a regenerative eco-village near Kårsta in Vallentuna: about 20 tiny houses, a shared greenhouse and a composting toilet system, on a few hectares of farmland we could buy. The land is outside any detaljplan. What are we missing, and how could we make this work?	PBL 4 kap. 2 §, MB 3 kap. 4 §, anläggningslag 1 §, FMH 13 §; Översiktsplan Vallentuna 2040 (Kårsta)
`develop-gotland-farm-cabins`	develop	We run a small farm on Gotland outside any village and want to add six guest cabins for farm stays, with a shared well and one shared wastewater system. Can you help us think this through: what would block it and what options do we have?	Region Gotland pages 'Vatten och avlopp i bygglovsärenden', Översiktsplan Vårt Gotland 2040 (2.5.1, 2.5.3); FMH 13 §

`develop-uppsala-farm-housing`	develop	I farm land in the countryside of Uppsala kommun and would like to build three or four houses for farm workers and family members, and maybe sell a couple of plots later. What do I need to think about, and what is the best way to go about it?	Översiktsplan Uppsala 2016, Del A (landsbygd); PBL 9 kap. 74 §; MB 3 kap. 4 §

Generated cases (28 in 28 other municipalities, `agentic/cases_from_corpus.py`): explain = one municipal guidance page (permits, sewage, fees, shoreline, planbesked); propose = the comprehensive-plan passages of a municipality that name places for growth (utvecklingsområden, prioritised towns, LIS areas); develop = one of 6 project ideas inspired by the vision (eco-village on farmland, tiny houses on a farm, barn or school turned into co-housing, farm-stay cabins, market garden, co-housing by a lake) + the municipality's rural and water/sewage passages + the core statutes. The drafting model is **MiMo v2.6 Flash**, another model family than the agent, so the reference is not written by the model under test; it must give each key point (and each place) with a quote of 6–15 words of one source, and only points whose quote is found in that source (longest run of ≥ 5 words, as for the MÖD cases) are kept. Checks: a propose question must not name the places; places reduced to one name each (area codes, roads and plan names dropped, the municipality itself excluded); develop questions must not mention the rules (finding them is the task). A manual review (8 corrections in `REVIEW`: misleading or mislabelled points, anchors re-read from the source) is applied on every rebuild. Split by task (seeds + generated): develop 13, explain 13, propose 12. Prompt for develop cases:

```
You build test cases for Pyrmit, an assistant on Swedish planning and land law used by landowners, farmers,
architects, developers and eco-village groups. You get numbered sources [S1], [S2]... (Swedish municipal documents
and
statutes). Everything in the reference must come from these sources: each item carries a quote of 6-15 consecutive
words copied exactly from one source (Swedish, as written there, no ellipsis) and that source's number.
Return JSON only.
The user's idea: {idea}.
{"usable": true|false,   // false if the sources say too little that applies to this idea in this kommun
 "question": "<English, 2-4 sentences in the first person ('We want to...'): the user describes the idea in
<kommun>
               (a rough location type, no property, no plan or area names) and asks what they are missing and how
to
               make it work. Do NOT mention any rule, permit, plan type or constraint (no detaljplan, strandskydd,
               farmland protection, sewage permit...): finding them is the assistant's job.>",
 "key_points": [{"kind": "Obstacle"|"Lever"|"Ask", "point": "<English>", "quote": "...", "source": "..."}]
   // 4-6 points: 1-3 Obstacles (a rule or plan statement that could block the idea), 1-3 Levers (a way to make it
   // work: location, scale, plan route, shared facilities... that a source allows), and exactly one Ask (a fact
the
   // assistant must ask the user for; an Ask has quote "" and source "")
}
```

Leakage check (`--leakage`, no LLM): the rank of each expected passage in a plain search on the question as asked; a case whose evidence mostly ranks in the top 3 gives its answer away. None of the seeds does.

Scoring. Key points as before (0/1/2), develop points split into Obstacle / Lever / Ask; no verdict unless the case lists accepted ones; propose adds the places named (string match on the anchors, at least `min_anchors`) and the options the judge finds, sourced or not. Judge prompt for propose (`agentic/bench.py` `TASK_JUDGE_INSTRUCTIONS`):

```
You grade answers of a Swedish planning assistant that proposes where or what a user could build, against a
reference.
Return JSON only: {"verdict": "<...|none>", "proposals": [{"place": "<as the answer names it>", "sourced":
true|false}], "points": [<0|1|2> per key point, in order], "unsupported_claims": <int>, "comment": "<one
sentence>"}.
- proposals: each distinct option (place or kind of project) the answer proposes. sourced = the cited sources given
  below show that the kommun's plans designate or support this place for this kind of development (not only that
the
  place exists), or the reference does.
- verdict: only if the reference lists acceptable verdicts, how the answer classifies the project (Feasible ->
feasible, Feasible with conditions -> conditional, Not feasible as described -> not_feasible, Unclear -> unclear);
"none" if it gives no classification.
- points: 2 = the key point is clearly stated, 1 = partially / vaguely, 0 = missing or contradicted.
- unsupported_claims: number of specific rules/numbers/facts in the answer that are stated neither in the reference
nor
  in the cited sources given below, or that contradict them. A claim the cited sources state is supported even when
  the reference does not mention it (the reference is not exhaustive).
```

"In Västervik, how do I report a neighbour who is building without permission or has a badly overgrown plot — who do I contact, and what do I have to write?"

Task: explain. **Source**: Västervik: Hur du anmäler - Västerviks kommun

Expected key points:

- Reports of illegal building or an unkempt plot go to the miljö- och byggnadskontoret, and can be made by phone, letter or email, though the kommun prefers written reports to avoid misunderstandings.
- The report should give the property/address, what was built, when it happened, a detailed description with measurements and distances to boundaries, plus photos and a map marking the spot; you cannot report something you merely suspect is being planned.
- You may stay anonymous, but if you do you receive neither the decision nor any information about the case — to be anonymous, send a letter without your name and contact details.
- Typical reportable matters include building work started without a startbesked, a building put into use before slutbesked, use differing from the building permit, or something built that differs from what was approved.
- After more than ten years the plan- och byggnadsnämnden cannot demand correction of illegal building (it is time-barred, not legalised), but the kommun can still act against a plot or structure kept in an unkempt state.

"We are a landowner group looking to build a cluster of 15-20 small houses in Botkyrka. Where in the kommun would the översiktsplan allow such a project, and what approvals, studies and infrastructure would we need to get it through?"

Task: propose. **Source**: Botkyrka: översiktsplan growth passages

Designated places (anchors): Grödinge, Vårsta, Tegelvreten, Eldtomta (at least 2 expected)

Expected key points:

- Single build applications are decided case by case, and the kommun's default is to refuse new housing outside the designated locations.
- A project with several properties must go together with a detailed development plan (detaljplan) and an extension of municipal water and sewer.
- Where no municipal connections exist, approved private water and wastewater solutions are required.
- Suitability is tested against ancient monuments, shoreline protection, power lines, railway noise and valuable nature, and buildings must fit the cultural landscape.

"We want to set up an eco-village of 15 to 25 homes with shared gardens and a single shared wastewater system, on farmland just outside a village in Upplands Väsby. What are we missing before we can move ahead, and how should we go about making this work?"

Task: develop. **Source**: Upplands Väsby: rural / VA passages + statutes; idea: an eco-village of 15-25 homes with shared gardens and one shared wastewater system, on farmland outside a village

Expected key points:

- Obstacle: New housing on valuable agricultural land in the countryside is restricted, so a 15-25 home scheme on farmland runs into a hard limit.
- Obstacle: Municipal policy concentrates growth inside the urban area, strongly limiting any new scattered dwellings on the countryside.
- Obstacle: A shared wastewater installation that toilets are connected to requires an individual permit from the municipal committee, which the group must plan for.
- Lever: Small-scale new housing in the countryside can still be examined on a small number of sites, if they sit in existing village formations with functioning communication routes.
- Lever: Shared facilities serving several properties can be established as a common facility, which suits a single shared wastewater plant for many homes.
- Ask: Which existing settlement group and communication route outside the village is the site attached to, and how high is the agricultural land grading there?

7.10 Verifying the system

The bugs found on 2026-10-22 (benchmark report, 2.10) had passed the unit tests and the benchmarks: the tests fed the tools the inputs we imagined, the benchmarks scored the final answer (the agent worked around failed calls), and the data were judged on totals (a median hid twenty empty municipalities). Four checks now look at what the system really does; they cost nothing (no LLM call).

Check	What it catches	When	Latest result
Data checks `python -m corpus.check`	Municipalities with no or few documents, at a cap, or that lost half their items since the last check; PDFs no manifest references; garbled titles, replacement characters, script artefacts, off-topic or duplicate documents, the OCR backlog; statute sections out of order and chapter headings without chunks; chunks without a vector, a store behind `chunks.jsonl`; plans without any document. Exit code 1 on an error.	After every rebuild	Last run 2026-10-24: 1 errors, 7 warnings.
Tool health `agentic.health`	Tool calls that returned nothing usable, per tool and reason (`ToolResult.error`: unknown_id, bad_args, not_found, no_data, exception), with a threshold per tool	Printed by every benchmark run; table in the benchmark report	Benchmark report, 2.10
Replay `python -m agentic.replay`, `tests/test_replay.py`	Every distinct call the agent made in the stored runs, replayed on the current corpus: no exception, a read of an id the corpus has succeeds whatever the form it was written in, a law_section of a section the corpus has succeeds	In the test suite (sample of 1,500 calls) and after a rebuild	Last run: 11,350 distinct stored calls, 0 failed on something the corpus has, 428 on ids that left the corpus since.
Coverage `python -m agentic.coverage`	What the corpus misses: the share of the plans a municipality lists (fetched live) whose documents are in the corpus	After a crawl; re-fetch the reference when sites change	Last run: 202 plans of 13 municipalities, 76% with at least one PDF in the corpus, 57% of their PDFs; 16 municipalities publish their plans only in a map or search application (benchmark report, 2.11).

The app's **Pipeline** view shows these measurements per corpus-building step (collect, filter, OCR, index, plan registry, search tools), each with what the step does, how it is measured and the cases behind every check (`routers/pipeline.py`); the Benchmark view grades the answers end to end.

8. Results summary

Classic RAG and the agent at its default setting (exploration medium, answer medium) on the three sets; all configurations, charts and the reading of the results are in the benchmark report.

Case set	System	n	Verdict	Key points	Evidence seen	Evidence cited	Grounding	Passages seen	Latency	1st token	Cost / question
14 cases	RAG	28	1.00	0.59	0.60	0.48	0.90	20	16 s	11 s	$0.0017
	Agent (medium / medium)	28	1.00	0.77	0.88	0.54	0.89	69	31 s	24 s	$0.0071
MÖD	RAG	100	0.60	0.21	0.12	0.11	0.88	20	23 s	17 s	$0.0020
	Agent (medium / medium)	200	0.70	0.48	0.70	0.40	0.79	78	43 s	35 s	$0.0086
plankarta	RAG	40	0.35	0.31	0.47	0.28	0.81	20	24 s	18 s	$0.0020
	Agent (medium / medium)	40	0.62	0.67	1.00	0.90	0.68	66	107 s	99 s	$0.0110

The agent beats the classic RAG on key points and evidence on every set, and the gap grows with difficulty (MÖD and plan maps): a single search on the user's facts rarely reaches the deciding statute section or plan map, whereas the agent first works out which rule applies, then looks it up. The RAG is about twice as fast and four times cheaper, with as high a grounding (it says less). Within the agent, the exploration effort is the setting that matters; map reading lifts the verdict on plan-map questions; refused projects remain the agent's weak spot (it leans towards “feasible with conditions”).

Explain / propose / develop (section 7.9; MiMo judge; “Agent, propose fix” = the propose guide and forced `answer_ready` of 2026-10-21):

Task	System	n	Key points	Evidence seen	Evidence cited	Unsupported	Mode right	Task-specific	Latency	Cost / question
explain	RAG	13	0.63	0.83	0.59	0.9	–	–	13 s	$0.0013
	Agent	13	0.69	1.00	0.84	0.2	0.92	–	33 s	$0.0042
propose	RAG	12	0.41	0.12	0.07	2.1	–	places 0.39, 4.2 options, 0.61 sourced	21 s	$0.0020
	Agent	12	0.53	0.64	0.22	1.9	0.42	places 0.68, 3.8 options, 0.98 sourced	75 s	$0.0150
	Agent, propose fix	12	0.62	0.51	0.15	1.5	1.00	places 0.88, 4.6 options, 0.98 sourced	85 s	$0.0148
develop	RAG	13	0.47	0.12	0.12	0.4	–	obstacles 0.44, levers 0.42, asks 0.62	24 s	$0.0021
	Agent	13	0.70	0.74	0.56	1.4	0.92	obstacles 0.68, levers 0.69, asks 0.69	63 s	$0.0114

9. Limitations and roadmap

- **Evaluation**: the Luna judge agrees with a second model family (section 7.7); key points of the MÖD, plankarta and generated sets are written by an LLM and only a sample was reviewed (by the developers, not a planning expert). Next: expert review of a sample.
- **Scope of the benchmarks**: explaining, proposing places and developing an idea are now measured in a single turn (section 7.9); multi-turn guidance, comparing municipalities and arguing a case to the municipality are not.
- **Data**: coverage depends on each municipality's website; no property → plan link; no spatial layers; no provenance dates (section 4). Next: provenance metadata, scheduled incremental refresh, Lantmäteriet and county GIS data.
- **Plans**: provisions are extracted as text; which map area carries which code needs a vision model on the plan map.
- **Agent**: verdict calibration on refusals (prompt), default exploration effort medium, a fast low-effort mode.

Next steps (recommended order, after the optimisation round of 2026-10-21, benchmark report section 2.9):

1. **Confirm the adopted prompt** ("complete answer"): a second pass on the 52 dev cases, and the MÖD sample and plankarta set, which the optimisation round did not run (≈ $1). A +0.05 gain from one pass is close to the noise.
2. **Check step on the draft answer**: before streaming, check each claim against the chunk it cites and drop or flag what the chunk does not state, to bring the unsupported claims down (≈ 1.5–2 per answer for the Luna judge) without losing the coverage the "complete answer" instruction gained. Measure with both judges and 2 passes.
3. **A stronger model for the writing only**: the gains are in the answer, not in the exploration; test MiMo v2.6 Pro or GPT-6 Sol on the final turn only (Luna keeps exploring), with the check step, on a subset first: Sol costs ≈ 20× Luna per token.
4. **Expert review**: a Swedish planning expert scores a sample of answers and reviews the generated references (key points written by an LLM, reviewed only by the developers).
5. **Benchmark coverage**: "I have this land, what could I do with it?" (place fixed, project open); multi-turn cases (iterating on a project with the user); the localisation work as a `locate` task in the same case format, after the merge.
6. **Latency**: two of 52 answers waited over 6 minutes on a stalled provider stream: retry a stream that stalls, and bring propose (≈ 80 s) closer to the other modes.

Appendix: commands and files

```
cd backend
python -m corpus.municipalities && python -m corpus.laws && python -m corpus.crawl
python -m corpus.ocr && python -m corpus.extract && python -m corpus.chunk
python -m localrag.index --model st-arctic-l-v2 && python -m localrag.store --model st-arctic-l-v2 && python -m
corpus.plans
python -m localrag.eval --models st-arctic-l-v2
python -m corpus.caselaw && python -m agentic.cases_from_caselaw
python -m agentic.bench --systems agent --store disk --explore-effort medium --effort medium \
    --cases-file cases_caselaw.jsonl --sample 50 --repeats 2 --workers 3
python -m agentic.grounding data/agentic/bench-....json
python -m agentic.cases_from_corpus --per-task 10          # explain / propose / develop cases (MiMo v2.6 Flash)
python -m agentic.bench --cases-file cases_tasks.jsonl --store disk --systems rag agent \
    --explore-effort medium --effort medium --judge-model xiaomi/mimo-v2.6-flash --workers 4
python -m agentic.rejudge data/agentic/bench-....json --sample 60   # second judge, agreement
python -m agentic.report && python -m agentic.report_technical
```